Guia de Estudo: Bibliotecas Estáticas e GNU Make

Módulos 8 a 12 — da biblioteca `libft.a` às funções avançadas do Make

Nível: intermediário | Foco: projetos em C (estilo Libft, 42)

Sumário

Módulo 8 - Bibliotecas estáticas .a, ar e ranlib

Módulo 9 - Regras implícitas, Pattern Rules e %

Módulo 10 - wildcard, patsubst, addprefix e addsuffix

Módulo 11 - filter, filter-out, sort, strip e foreach

Módulo 12 - call, eval, origin, value e flavor

Gabarito dos exercícios e das avaliações

Cola rápida (cheat sheet)

Notas de revisão deste guia

Ao organizar o material, ajustei três pontos para que os exemplos funcionem de fato: (1) no Módulo 11, redefinir `SRC = $(sort $(SRC))` com `=` gera recursão infinita; use `:=`. (2) No Módulo 12, `echo $(value B)` entrega `$(A)` ao shell, que tentaria executar um comando chamado A; use aspas simples. (3) Nos blocos de Makefile deste PDF as linhas de comando aparecem indentadas com espaços, mas no arquivo real elas **precisam começar com TAB**.

Módulo 8 - Bibliotecas estáticas .a, ar e ranlib

Objetivos do módulo

• O que é uma biblioteca estática • Diferença entre .o, .a e executável • ar rcs e ranlib • Criar libft.a e usá-la em outro programa • Makefile para biblioteca

1. O que é uma biblioteca estática?

Em vez de distribuir todos os `.c`, você junta os arquivos compilados (`.o`) em um único arquivo `.a` (archive), como `libft.a`. A biblioteca pode ser reutilizada por vários programas.

```
ft_strlen.c -> ft_strlen.o --\
ft_memset.c -> ft_memset.o ---+--> libft.a  (biblioteca estática)
ft_bzero.c  -> ft_bzero.o  --/
```

Arquivo	O que representa
.c	Código-fonte C
.h	Declarações e interfaces
.o	Código compilado de um .c
.a	Archive contendo vários objetos compilados
Executável	Programa final pronto para execução

Uma biblioteca `.a` **não é executada**: ela é usada na etapa de linkedição de um programa.

2. O comando ar

`ar` cria e manipula archives. Normalmente já vem instalado (pacote `binutils` no Ubuntu). Teste com `ar --version`.

2.1 Criando uma biblioteca manualmente

```
# ft_soma.c
int ft_soma(int a, int b)
{
    return (a + b);
}
# ft_subtrai.c e ft_multiplica.c seguem o mesmo modelo (a - b, a * b)
```

```
cc -Wall -Wextra -Werror -c ft_soma.c -o ft_soma.o
cc -Wall -Wextra -Werror -c ft_subtrai.c -o ft_subtrai.o
cc -Wall -Wextra -Werror -c ft_multiplica.c -o ft_multiplica.o
```

3. Criando a biblioteca com ar rcs

```
ar rcs libcalculadora.a ft_soma.o ft_subtrai.o ft_multiplica.o
```

Parte	Significado
ar	Programa que manipula archives
r	Insere ou substitui arquivos no archive
c	Cria o archive se não existir
s	Cria ou atualiza o índice de símbolos

Parte	Significado
libcalculadora.a	Nome da biblioteca
.o	Objetos incluídos

3.1 Inspecionando a biblioteca

```
ar -t libcalculadora.a     # lista os membros (os .o)
nm libcalculadora.a        # lista os símbolos

ft_soma.o:
0000000000000000 T ft_soma     # T = símbolo definido na seção de código
```

4. O que é ranlib?

`ranlib` cria ou atualiza o **índice de símbolos**, que ajuda o linker a achar funções dentro da biblioteca. Como `ar rcs` já faz isso (letra `s`), normalmente não é necessário chamar `ranlib` à parte. Em Makefiles antigos você verá:

```
ar rc libcalculadora.a $(OBJ)
ranlib libcalculadora.a        # forma antiga

ar rcs libcalculadora.a $(OBJ) # forma moderna e comum
```

5. Usando a biblioteca em outro programa

```
# calculadora.h
#ifndef CALCULADORA_H
# define CALCULADORA_H

int ft_soma(int a, int b);
int ft_subtrai(int a, int b);
int ft_multiplica(int a, int b);

#endif
```

```
# main.c
#include <stdio.h>
#include "calculadora.h"

int main(void)
{
    printf("Soma: %d\n", ft_soma(10, 5));
    printf("Subtracao: %d\n", ft_subtrai(10, 5));
    printf("Multiplicacao: %d\n", ft_multiplica(10, 5));
    return (0);
}
```

```
cc main.c -L. -lcalculadora -o programa
./programa
Soma: 15
Subtracao: 5
Multiplicacao: 50
```

Opção	Significado
-L.	Procura bibliotecas no diretório atual

Opção	Significado
-lcalculadora	Procura libcalculadora.a (não escreva lib nem .a)
-o programa	Define o nome do executável

Regra de nomenclatura: `lib` + `calculadora` + `.a`. Ao usar `-lcalculadora`, o linker procura `libcalculadora.a`.

6. Makefile para biblioteca estática

```
NAME = libcalculadora.a

CC = cc
CFLAGS = -Wall -Wextra -Werror

AR = ar
ARFLAGS = rcs

SRC = ft_soma.c \
      ft_subtrai.c \
      ft_multiplica.c

OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(AR) $(ARFLAGS) $(NAME) $(OBJ)

%.o: %.c calculadora.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Por que não usar `cc` para criar a biblioteca? São ferramentas diferentes: `cc` compila e faz linkedição; `ar` cria e manipula archives; `ranlib` indexa símbolos.

7. Recompilação e a armadilha do ar rcs

Se você altera só `ft_soma.c`, o Make recompila apenas `ft_soma.o` e atualiza `libcalculadora.a`. Isso economiza tempo em projetos grandes.

> **Atenção**
>
> `ar rcs` **adiciona ou atualiza** membros, mas não remove os antigos. Se você tirar `ft_multiplica.c` de `SRC`, o `ft_multiplica.o` pode continuar dentro da `.a`. Para garantir uma biblioteca só com os objetos atuais, rode `make re` (apaga a antiga e cria uma nova).

8. Aplicação: Libft

```
SRC = ft_strlen.c \
```

```
      ft_memset.c \
      ft_bzero.c \
      ft_memcpy.c \
      ft_memmove.c \
      ft_strlcpy.c \
      ft_strlcat.c
# make -> gera libft.a      make fclean -> remove tudo
```

9. Resumo de comandos

Comando	Utilidade
cc -c arquivo.c -o arquivo.o	Compila um objeto
ar rcs lib.a arquivos.o	Cria ou atualiza biblioteca
ar -t lib.a	Lista conteúdo
nm lib.a	Mostra símbolos
ranlib lib.a	Atualiza índice de símbolos
cc main.c -L. -lmylib	Linka biblioteca
make clean / fclean / re	Remove objetos / objetos e biblioteca / reconstrói tudo

10. Mini avaliação (5 questões)

1. Qual comando cria uma biblioteca estática? (gcc / ar rcs / make clean / ranlib somente)

2. Qual comando lista os objetos dentro de uma biblioteca? (ar -t / nm -c / cc -t / make -l)

3. O que faz `-lcalculadora`? (compila calculadora.c / procura libcalculadora.a / executa calculadora / cria um .o)

4. O `ar rcs` atualiza automaticamente o índice de símbolos? (sim / não)

5. Qual alvo garante recriação limpa da biblioteca? (clean / all / fclean / re)

As respostas estão no Gabarito.

Módulo 9 - Regras implícitas, Pattern Rules e %

Objetivos do módulo

• Regras implícitas e Pattern Rules • O símbolo % • Variáveis automáticas $@, $<, $^, $* • Projetos com diretórios src/, include/, obj/ • Order-only prerequisites

1. O problema das regras repetitivas

Escrever uma regra por arquivo funciona, mas é repetitivo (imagine 50 funções):

```
ft_strlen.o: ft_strlen.c
    cc -Wall -Wextra -Werror -c ft_strlen.c -o ft_strlen.o
ft_memset.o: ft_memset.c
    cc -Wall -Wextra -Werror -c ft_memset.c -o ft_memset.o
# ... e assim por diante
```

2. Pattern Rule e o símbolo %

```
%.o: %.c
    cc -Wall -Wextra -Werror -c $< -o $@
```

O `%` representa uma parte variável do nome, igual no alvo e na dependência. Ao construir `ft_strlen.o`, o Make deduz `% = ft_strlen` e procura `ft_strlen.c`. Uma única regra cobre todos os arquivos.

3. Variáveis automáticas

Variável	Significado
$@	Nome do alvo
$<	Primeira dependência
$^	Todas as dependências, sem duplicatas
$+	Todas as dependências, incluindo duplicatas
$?	Dependências mais novas que o alvo
$*	Stem: a parte que casou com o %
$(@D) / $(@F)	Diretório / nome do arquivo do alvo
$(<D) / $(<F)	Diretório / nome do arquivo da primeira dependência

```
programa: main.o matematica.o
    cc $^ -o $@         # = cc main.o matematica.o -o programa

%.o: %.c                 # ao construir ft_strlen.o:
    cc -Wall -Wextra -Werror -c $< -o $@
# $@ = ft_strlen.o   $< = ft_strlen.c   $* = ft_strlen
```

O `@` antes de um comando (`@echo ...`) faz o Make executar sem imprimir o próprio comando.

4. Regras implícitas x Pattern Rule x regra explícita

Conceito	Explicação
Regra implícita	Já embutida no GNU Make (ele sabe compilar .c em .o usando $(CC) e $(CFLAGS))

Conceito	Explicação
Pattern Rule	Escrita por você usando %
Regra explícita	Definida para um alvo específico, ex.: main.o: main.c main.h

Dá para depender das regras implícitas, mas declarar as suas dá mais controle sobre flags, headers, diretórios e dependências.

5. Projeto simples com %

```
NAME = calculadora
CC = cc
CFLAGS = -Wall -Wextra -Werror

SRC = main.c ft_soma.c ft_subtrai.c ft_multiplica.c
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $^ -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

6. Pattern Rules com diretórios e order-only

```
projeto/
  Makefile
  src/      main.c  ft_soma.c  ft_subtrai.c
  include/  calculadora.h
  obj/
```

```
NAME = calculadora
CC = cc
CFLAGS = -Wall -Wextra -Werror
CPPFLAGS = -Iinclude

SRC = src/main.c src/ft_soma.c src/ft_subtrai.c
OBJ = $(SRC:src/%.c=obj/%.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(OBJ) -o $@

obj/%.o: src/%.c | obj
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

obj:
    mkdir -p obj
```

```
clean:
    rm -rf obj

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

- `obj/%.o: src/%.c`: para `obj/ft_soma.o` o Make procura `src/ft_soma.c` (`% = ft_soma`).
- `-Iinclude` indica onde procurar headers (vai em `CPPFLAGS`).
- `| obj` é uma **order-only prerequisite**: garante que a pasta exista antes de compilar, mas não força recompilação só porque a pasta mudou de data.

7. Exercícios

- **Ex. 1:** Makefile com main.c, ft_soma.c, ft_subtrai.c, ft_multiplica.c usando `%.o: %.c`, `$<` e `$@`, gerando `calculadora` com all, clean, fclean e re.
- **Ex. 2:** Em `programa: main.o utils.o calculadora.o` com `cc $^ -o $@`, quanto valem `$@`, `$^`, `$<` e, em `%.o: %.c`, o stem de `utils.o`?
- **Ex. 3:** Estrutura src/, include/, obj/ com Pattern Rules, `-Iinclude`, criação automática de obj/ e clean.

8. Mini avaliação (5 questões)

1. O que significa % em uma Pattern Rule? (comentário / parte variável do nome / variável de ambiente)
2. Em `%.o: %.c`, qual o valor de $@ ao compilar ft_strlen.o? (ft_strlen.c / ft_strlen / ft_strlen.o)
3. Qual variável representa a primeira dependência? ($@ / $^ / $<)
4. Qual variável representa o stem de ft_strlen.o? ($* / $@ / $?)
5. Qual o benefício das Pattern Rules? (eliminar .c / evitar regras repetidas / substituir o compilador)

Módulo 10 - wildcard, patsubst, addprefix e addsuffix

Objetivos do módulo

• Descobrir arquivos automaticamente • Transformar listas e caminhos • Makefile automático para C

1. O problema

Manter `SRC` e `OBJ` escritos à mão obriga a editar duas listas a cada novo arquivo. As funções do Make resolvem isso.

2. wildcard: encontrar arquivos

```
SRC = $(wildcard *.c)          # todos os .c do diretório atual
SRC = $(wildcard src/*.c)      # todos os .c de src/
```

Atenção: `wildcard` encontra arquivos **que já existem**; não serve para declarar arquivos que ainda serão criados.

3. patsubst: substituir padrões

```
$(patsubst padrão,substituição,texto)

SRC = main.c utils.c player.c
OBJ = $(patsubst %.c,%.o,$(SRC))
# main.o utils.o player.o

SRC = src/main.c src/player.c src/utils.c
OBJ = $(patsubst src/%.c,obj/%.o,$(SRC))
# obj/main.o obj/player.o obj/utils.o
```

4. addprefix e addsuffix

```
SRC = main.c player.c utils.c
OBJ = $(addprefix obj/,$(SRC:.c=.o))
# obj/main.o obj/player.o obj/utils.o

FILES = main player utils
SRC = $(addsuffix .c,$(FILES))
# main.c player.c utils.c
```

Função	Utilidade
wildcard	Encontrar arquivos
patsubst	Substituir padrões
addprefix	Adicionar prefixo a cada elemento
addsuffix	Adicionar sufixo a cada elemento

5. Projeto prático: Makefile automático

```
NAME = programa
CC = cc
```

```
CFLAGS = -Wall -Wextra -Werror
CPPFLAGS = -Iinclude

SRC = $(wildcard src/*.c)
OBJ = $(patsubst src/%.c,obj/%.o,$(SRC))

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(OBJ) -o $(NAME)

obj/%.o: src/%.c | obj
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

obj:
    mkdir -p obj

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Teste: rode `make`, depois `touch src/teste.c` e `make` de novo; o novo arquivo é reconhecido sozinho. Para inspecionar variáveis: `make -pn | grep '^SRC'`.

6. Exercícios

- **Ex. 1:** variável que encontre todos os `.c` de `src`.
- **Ex. 2:** transformar `src/main.c src/game.c src/player.c` em `obj/main.o obj/game.o obj/player.o` (patsubst).
- **Ex. 3:** transformar `main.o game.o player.o` em `obj/main.o obj/game.o obj/player.o` (addprefix).
- **Ex. 4:** transformar `main game player` em `main.c game.c player.c` (addsuffix).
- **Ex. 5 (desafio):** Makefile que ache os .c de src, gere os .o em obj, compile `game` com -Wall -Wextra -Werror, tenha all/clean/fclean/re e não recompile desnecessariamente.

Módulo 11 - filter, filter-out, sort, strip e foreach

Objetivos do módulo

• Filtrar e remover elementos de listas • Ordenar e eliminar duplicados • Normalizar espaços • Percorrer listas com foreach

1. filter e filter-out

```
FILES = main.c main.o utils.c utils.o player.c player.o
CFILES = $(filter %.c,$(FILES))        # main.c utils.c player.c
OFILES = $(filter %.o,$(FILES))        # main.o utils.o player.o
RES    = $(filter %.c %.h,$(FILES))    # vários padrões

FILES = main.c debug.c test.c utils.c
SRC = $(filter-out debug.c test.c,$(FILES))    # main.c utils.c

SRC = main.c utils.c test_main.c test_utils.c
PRODUCTION = $(filter-out test_%.c,$(SRC))
TESTS      = $(filter test_%.c,$(SRC))
```

`filter` seleciona o que casa com o padrão; `filter-out` faz o contrário.

2. sort

```
FILES = player.c main.c utils.c main.c player.c
RESULT = $(sort $(FILES))      # main.c player.c utils.c

SRC = $(sort $(wildcard src/*.c))
```

`sort` ordena alfabeticamente **e remove duplicados**. Não preserva a ordem original.

3. strip

```
FILES =     main.c      utils.c       player.c
CLEAN = $(strip $(FILES))      # main.c utils.c player.c

ifeq ($(strip $(SRC)),)
$(error SRC está vazio!)
endif
```

`strip` remove espaços das pontas e reduz sequências internas a um único espaço. Útil para normalizar listas e testar variáveis vazias.

4. foreach

```
$(foreach variável,lista,expressão)

FILES = main utils player
OBJ = $(foreach file,$(FILES),obj/$(file).o)    # obj/main.o obj/utils.o obj/player.o

# Equivalente mais simples:
OBJ = $(addprefix obj/,$(addsuffix .o,$(FILES)))

# foreach + wildcard em vários diretórios
DIRS = src src/core src/utils
SRC = $(foreach dir,$(DIRS),$(wildcard $(dir)/*.c))
```

Regra prática: use `foreach` quando a transformação por elemento for mais complexa que um simples prefixo/sufixo.

5. Combinando funções

```
SRC := $(strip $(sort $(filter-out src/test_%.c,$(wildcard src/*.c))))
```

Ordem: `wildcard` acha os arquivos, `filter-out` remove os de teste, `sort` ordena e elimina duplicados, `strip` normaliza espaços.

Cuidado com a recursão

Escrever `SRC = $(sort $(SRC))` (com `=`) faz a variável se referenciar a si mesma, e o Make aborta com 'Recursive variable references itself'. Para reatribuir a partir do valor anterior, use `:=` (expansão imediata).

6. Projeto: Makefile com filtros

```
NAME = game
CC = cc
CFLAGS = -Wall -Wextra -Werror
CPPFLAGS = -Iinclude

ALL_SRC := $(wildcard src/*.c)
SRC     := $(sort $(filter-out src/test_%.c,$(ALL_SRC)))
OBJ     := $(patsubst src/%.c,obj/%.o,$(SRC))

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(OBJ) -o $(NAME)

obj/%.o: src/%.c | obj
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

obj:
    mkdir -p obj

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Função	O que faz
filter	Seleciona elementos
filter-out	Remove elementos
sort	Ordena e elimina duplicados
strip	Normaliza espaços
foreach	Percorre uma lista executando uma expressão

7. Exercícios

- **Ex. 1:** de `main.c utils.o player.c game.o`, criar `SRC` só com os `.c`.
- **Ex. 2:** de `main.c debug.c player.c test.c utils.c`, remover `debug.c` e `test.c`.
- **Ex. 3:** transformar `b.c a.c c.c a.c b.c` em lista ordenada sem duplicados.
- **Ex. 4:** criar variável com espaços extras e normalizar com `strip`.
- **Ex. 5:** de `main utils player`, gerar `src/main.c src/utils.c src/player.c` (foreach).
- **Ex. 6 (desafio):** Makefile que ache os .c, exclua `test_*.c`, ordene, gere .o em obj/, compile `game` e tenha all/clean/fclean/re.

Módulo 12 - call, eval, origin, value e flavor

Objetivos do módulo

• Criar funções personalizadas (call) • Gerar regras dinamicamente (eval) • Investigar variáveis (origin, value, flavor)

1. call: funções personalizadas

```
SAUDACAO = Olá, $(1)!
RESULTADO = $(call SAUDACAO,Linux)
all:
    @echo $(RESULTADO)          # Olá, Linux!

OBJETO = obj/$(1).o
OBJ = $(call OBJETO,main) $(call OBJETO,player)    # obj/main.o obj/player.o
```

`$(1)`, `$(2)`... são os argumentos. Use `call` para evitar repetição e organizar Makefiles grandes. (Fazer contas com `shell expr` serve só de demonstração: o Make não é uma linguagem de cálculo.)

2. eval: gerar código Make dinamicamente

```
PROGRAMAS = game editor server

define REGRA
$(1):
    @echo Compilando $(1)
endef

$(foreach programa,$(PROGRAMAS),$(eval $(call REGRA,$(programa))))
# make game   -> Compilando game
```

- `define ... endef` cria o modelo da regra.
- `call` substitui `$(1)`.
- `eval` interpreta o texto resultante como Makefile.
- `foreach` repete para cada programa.

Dois níveis de expansão

Dentro de um `define` usado com `eval`, escreva `$$@`, `$$(CC)`, `$$<` para que essas variáveis sobrevivam à primeira expansão e sejam resolvidas só quando a regra rodar. Sempre se pergunte em que momento cada variável será expandida.

3. origin: de onde veio a variável?

Resultado	Significado
undefined	Variável não definida
default	Padrão do Make
environment	Veio do ambiente
environment override	Do ambiente, prevalecendo sobre o Makefile
file	Definida em um Makefile
command line	Definida na linha de comando

Resultado	Significado
override	Definida com a diretiva override
automatic	Variável automática, como $@

```
CC = cc
all:
    @echo CC: $(CC)
    @echo Origem: $(origin CC)

make            -> CC: cc      Origem: file
make CC=clang   -> CC: clang   Origem: command line
```

4. value: conteúdo literal, sem expandir

```
A = Linux
B = $(A)
all:
    @echo Normal: $(B)            # Linux
    @echo 'Literal: $(value B)'   # Literal: $(A)
```

Use aspas simples no `echo`: sem elas o shell interpretaria `$(A)` como substituição de comando.

5. flavor: tipo de expansão

```
A = Linux
B = $(A)       # recursiva: expandida quando usada
C := $(A)      # simples: expandida na hora

A = Ubuntu
# $(B) -> Ubuntu   $(flavor B) -> recursive
# $(C) -> Linux    $(flavor C) -> simple
```

Definição	Comportamento	flavor
A = valor	Expansão quando utilizada	recursive
A := valor	Expansão imediata	simple
A ?= valor	Define só se ainda não estiver definida	depende da definição
A += valor	Acrescenta conteúdo	depende da variável original

6. Projeto: Makefile dinâmico

```
CC = cc
CFLAGS = -Wall -Wextra -Werror
PROGRAMAS = game editor server

define CRIAR_PROGRAMA
$(1): src/$(1).c
    $$(CC) $$(CFLAGS) $$< -o $$@
endef

$(foreach programa,$(PROGRAMAS),$(eval $(call CRIAR_PROGRAMA,$(programa))))

all: $(PROGRAMAS)

clean:
    rm -f $(PROGRAMAS)
```

```
re: clean all

info:
    @echo "Compilador: $(CC)"
    @echo "Origem: $(origin CC)"
    @echo "Tipo: $(flavor CC)"
    @echo "Flags: $(value CFLAGS)"

.PHONY: all clean re info
```

Cada executável vem de um único `.c`; é uma demonstração de geração dinâmica, não uma estrutura completa para projetos com vários arquivos.

Recomendação

Não use `eval` em todo Makefile. Para projetos comuns em C, Pattern Rules (`%.o: %.c`) são mais simples e fáceis de manter.

Função	Finalidade
call	Executar funções personalizadas
eval	Interpretar código Make dinamicamente
origin	Descobrir a origem de uma variável
value	Obter o conteúdo literal de uma variável
flavor	Identificar o tipo de expansão

7. Exercícios

- **Ex. 1:** função `CAMINHO` tal que `$(call CAMINHO,src,main.c)` gere `src/main.c`.
- **Ex. 2:** com `NOME = Luiz`, use `eval` para criar `USUARIO` com o valor Luiz.
- **Ex. 3:** mostrar a origem de CC, CFLAGS e NAME e testar `make CC=clang`.
- **Ex. 4:** mostrar a diferença entre `$(B)` e `$(value B)` com `A = Linux`, `B = $(A)`.
- **Ex. 5:** descobrir `$(flavor A)` e `$(flavor B)` para `A = Linux` e `B := Linux`.
- **Ex. 6 (desafio):** Makefile com game, editor e server usando call, foreach e eval, mais origin/flavor para CC e CFLAGS, com all/clean/re.

Gabarito dos exercícios e das avaliações

Módulo 8 - Mini avaliação

Questão	Resposta
1	ar rcs
2	ar -t
3	Procura libcalculadora.a
4	Sim (a opção s cria/atualiza o índice)
5	re (fclean + all, recria do zero)

Módulo 9 - Mini avaliação

Questão	Resposta
1	Uma parte variável do nome
2	ft_strlen.o
3	$<
4	$*
5	Evitar escrever regras repetidas para cada arquivo

Módulo 9 - Exercício 2

Pergunta	Resposta
$@	programa
$^	main.o utils.o calculadora.o
$<	main.o (primeira dependência)
Stem de utils.o em %.o: %.c	utils

Módulo 10 - Soluções

```
SRC = $(wildcard src/*.c)                          # Ex. 1
OBJ = $(patsubst src/%.c,obj/%.o,$(SRC))           # Ex. 2
OBJ = $(addprefix obj/,$(FILES))                   # Ex. 3
SRC = $(addsuffix .c,$(FILES))                     # Ex. 4
```

Módulo 11 - Soluções

```
SRC = $(filter %.c,$(FILES))                       # Ex. 1 -> main.c player.c
SRC = $(filter-out debug.c test.c,$(SRC))          # Ex. 2 -> main.c player.c utils.c
LIST = $(sort $(FILES))                            # Ex. 3 -> a.c b.c c.c
X = $(strip $(X_COM_ESPACOS))                      # Ex. 4
OUT = $(foreach f,$(FILES),src/$(f).c)             # Ex. 5
```

Módulo 12 - Soluções

```
CAMINHO = $(1)/$(2)                                # Ex. 1
```

```
NOME = Luiz
$(eval USUARIO = $(NOME))                                    # Ex. 2
# Ex. 4: $(B) -> Linux ; $(value B) -> $(A)
# Ex. 5: $(flavor A) -> recursive ; $(flavor B) -> simple
```

Os desafios (Ex. 5 do Módulo 10, Ex. 6 dos Módulos 11 e 12) são variações dos Makefiles completos mostrados em cada módulo.

Cola rápida (cheat sheet)

Biblioteca estática

```
cc -Wall -Wextra -Werror -c f.c -o f.o
ar rcs libx.a f.o g.o        # cria/atualiza (já indexa)
ar -t libx.a                 # lista membros
nm libx.a                    # lista símbolos
cc main.c -L. -lx -o prog    # linka com libx.a
```

Variáveis automáticas

```
$@ alvo   $< 1a dependência   $^ todas (sem duplicatas)   $* stem
$(@D) dir do alvo   $(@F) arquivo do alvo
```

Funções do Make

Função	Exemplo
wildcard	$(wildcard src/*.c)
patsubst	$(patsubst src/%.c,obj/%.o,$(SRC))
addprefix / addsuffix	$(addprefix obj/,$(OBJ)) / $(addsuffix .c,$(NAMES))
filter / filter-out	$(filter %.c,$(F)) / $(filter-out test_%.c,$(F))
sort / strip	$(sort $(F)) / $(strip $(F))
foreach	$(foreach d,$(DIRS),$(wildcard $(d)/*.c))
call / eval	$(call F,arg) / $(eval texto)
origin / value / flavor	$(origin CC) / $(value B) / $(flavor B)

Boas práticas

- Use `ar rcs` e, para reconstruir limpo, `make re`.
- Prefira Pattern Rules a regras repetidas; use `| obj` para criar diretórios.
- Use `:=` quando reatribuir uma variável a partir dela mesma.
- Linhas de comando de regra começam com TAB.
- Declare `.PHONY: all clean fclean re`.